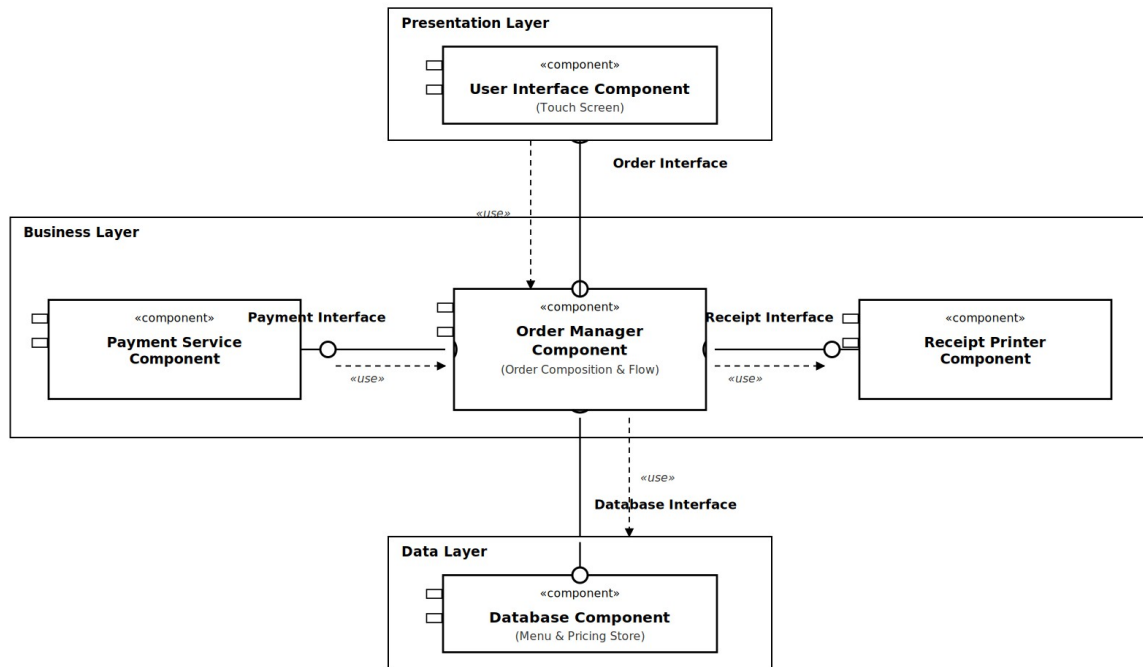


Lab 3: Component Diagram & Architectural Justification

Self-Service Coffee Kiosk System

Coffee Kiosk System — UML Component Diagram

Layered Architecture



○ Provided interface (ball) (Required interface (socket) - - -> «use» dependency □ «component»

5 components · 4 provided/required interfaces · «use» dependencies shown per layer

Architecture Selection: *"We chose Layered Architecture for the Self-Service Coffee Kiosk System."*

Architectural Choice

A four-layer architecture was selected: a Presentation Layer (User Interface Component), a Business Layer (Order Manager Component, Payment Service Component, Receipt Printer Component), and a Data Layer (Database Component). The Order Manager Component sits at the centre of the Business Layer and mediates all requests between the User Interface Component above it and the Payment, Receipt, and Database components around it, each accessed through a dedicated provided/required interface (Order Interface, Payment Interface, Receipt Interface, Database Interface).

Two Scenario-Related Reasons

- **Single, self-contained device:** The kiosk is one physical unit with a fixed, small scope (3 coffee types, 2 sizes, one payment method), so there is no independent-scaling or multi-team-deployment need that would justify Microservices' operational overhead. A Layered design gives the same clear separation of concerns with far less complexity.
- **Straightforward hardware and data integration:** The touch screen, receipt printer, and menu/pricing store are all local to the kiosk. Layering lets the Order Manager Component mediate cleanly between the User Interface Component, the Receipt Printer Component, and the Database Component without exposing hardware or storage details to the presentation layer.

Security Advantage

Because the User Interface Component can only reach the Payment Service Component indirectly through the Order Manager Component's Order Interface and Payment Interface, raw card data never has to pass through or be exposed to the presentation layer. This layer isolation narrows the attack surface for card-data handling and keeps payment logic confined to a single, auditable layer.

Performance Benefit

Menu and pricing data is read directly by the Order Manager Component from the local Database Component through the Database Interface, rather than over a network call as a Microservices split might require, so order totals and item lookups are computed with in-process, low-latency access, keeping the touch-to-confirmation time short for customers standing at the kiosk.